

Twenty-three Commits

What Claude Code has contributed to Informath, July–September 2026

Aarne Ranta

September 2026

This talk

- Informath: 899 commits since 2024, almost all by hand.
- **23** of them are co-authored by Claude (Claude Code, Anthropic's coding agent, working in a terminal in my checkout).
- For each task: **why** it was needed, the **prompt and interaction**, and the **outcome** — including what is still wrong.
- Sources: `git log`, the diffs, Claude Code session transcripts (September), session logs that Claude wrote itself (July).

These slides were themselves drafted by Claude from those sources — a 24th task, not committed yet.

Outline

Informath in five minutes

Part I: New languages

Part II: Godement's *Algebra*

Observations and the future

Informath in five minutes

The problem

Mathematics exists in two registers.

Formal

- Agda, Rocq, Lean, Dedukti
- machine-checkable
- unreadable to most humans

Informal

- English, French, German, Swedish, ...
- readable
- ambiguous, unverifiable

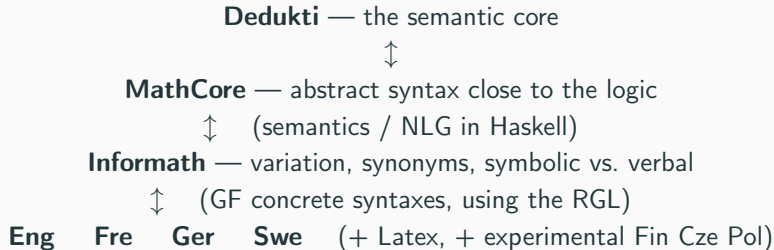
Informath translates between them in all directions: **informalization** (formal $\rightarrow$ informal), **autoformalization** (informal $\rightarrow$ formal), and informal $\rightarrow$ informal via the formal.

One statement, many languages

```
Dedukti: prop110 : (a : Elem Int) -> (c : Elem Int) ->  
  Proof (and (odd a) (odd c)) ->  
  Proof (forall Int (b => even (plus (times a b) (times b c))))).
```

- **En:** Let a and c be integers. Assume that a and c are odd. Then $ab + bc$ is even for all integers b .
- **Fr:** Soient a et c des entiers. Supposons que a et c sont impairs. Alors $ab + bc$ est pair pour tous les entiers b .
- **Sv:** Låt a och c vara heltal. Anta att a och c är udda. Då är $ab + bc$ jämnt för alla heltal b .

All generated from the Dedukti source, and Agda, Rocq and Lean as well.



- GF: Grammatical Framework; RGL: its Resource Grammar Library (40+ languages).
- The grammar is split into a language-independent *functor* and small per-language modules of words and function words.

Symbol tables: from constants to words

Each Dedukti constant gets a wording and/or a notation, given *by example*:

```
isGroup      : "#1 is a group"
isSubgroup   : "#1 is a subgroup of #2"
inv          : "the inverse of #2" |  $\$ \#2^{-1} \$$ 
cosetRel     : "#3 is congruent to #4 modulo #2"
```

- The example is *parsed* with the grammar into a lexical tree (NounPrepNoun2 subgroup_Noun of_Prep), which then works in any statement, in any language.
- $\$ \dots \$$ forms become LaTeX macros.
- Only shapes the grammar can parse are allowed — a recurring theme today.

How Claude worked

- Claude Code runs in a terminal, reads and edits the files, runs `gf`, `stack`, `RunInformath`, `pdflatex`.
- I give short prompts in English; Claude plans, works, reports.
- Commits only when asked; I push myself.
- It can spawn *subagents* for parallel subtasks (e.g. batches of words).
- Remembers preferences across sessions in a small memory file, e.g. “prefer symbolic expressions”.

The 23 commits as 11 tasks

	Task	Date	Commits
1	Finnish words and function words	10 July	2
2	A Czech grammar	10, 16 July	4
3	A Polish grammar	16 July	2
4	Completing the French Wikidata lexicon	22 July	1
5	Words for the Godement formalization	24–25 Sept	5
6	Clash-free macro names	24 Sept	1
7	Proof texts	25 Sept	4
8	Compressing hypotheses in proof texts	26 Sept	1
9	Formulas in standard logical notation	26 Sept	1
10	GodementSyntax: the language of definitions and theorems	26 Sept	1
11	GodementExamples: richer symbol tables	26 Sept	1

Part I: new languages. Part II: formalizing Godement's *Algebra*.

Part I: New languages

Task 1: Finnish — why

- A Finnish grammar existed as a *compilable template*, made by copying Swedish and English.
- It said `linking ... OK` — and it said `then_Adv = mkAdv "så"`.
- `VerbalConstantsFin`: an English copy with two Finnish words.
- `WikidataWordsFin` (2000+ terms) was a byte-identical copy of English and did not compile, so it was commented out:
a grammar that “worked” by excluding 2000 of its 2100 words.
- The question: can an LLM complete a GF grammar for a language it knows well, given the RGL?

Task 1: Finnish — prompt and interaction

Goal given (as recorded in Claude's session log):

“Complete the Finnish concrete grammars by replacing placeholder (mostly Swedish / English-copy) content with proper Finnish mathematical vocabulary.”

- The prompts themselves were not saved; this was the first experiment.
- Three layers: ~23 function words, ~120 core math words, 2051 Wikidata terms.
- The Wikidata terms went to **7 parallel subagents**, each returning an English–Finnish table; a Python script generated GF.
- Claude found why the lexicon did not compile, and dropped the dependency on the Finnish dictionary: Finnish *smart paradigms* suffice.
- My interventions: asking about prepositions; not accepting `linking ...` OK as proof of correctness.

Task 1: Finnish — outcome

9ac0431, ab82b63 (10 July)

- 2051 entries: 1216 nouns, 749 adjectives, 86 verbs; all enabled; links.
- *lyhde* (sheaf), *ydin* (kernel), *jaoton* (prime); *olkoon* $\times$ *ryhmä*; mkNoun "rengas" gives *kaikille renkaille* *R* with consonant gradation for free.

Remaining problems

- Five prepositions are loose words (mkPrep "yli") where Finnish uses cases: "They compile. They read acceptably. They are not right."
- Invented terms: *quiver* $\rightarrow$ *nuolikko*; capitalized *Abelin* next to *abelin*; 33 orphans, 15 duplicates inherited from English.
- No native review yet; moved to `grammars/next/`.

Task 2: Czech — why

- Czech: a new Informath language, and a test of a *weak* RGL.
- The template was Swedish down to the morphology: `mkN "typ" "typer"`.
- The Czech RGL was a stub: `SentenceCze` had 4 rules, no `mkV`, no `V3`; missing rules crashed only late: `NOT YET IMPLEMENTED: ComplVS`.
- Informath's functor calls the RGL by qualified names, so the gaps could not be worked around inside Informath.

Task 2: Czech — prompt and interaction

“Make `InformathCze.gf` compile, link, and produce Czech.”

- A loop: crash, write the missing RGL rule, rebuild the library — about 20 times. “Tirelessly.”
- Half of the session was spent in the RGL (commits there too).
- 2003 terms by **8 subagents**, with French and German as hints.
- Asked to audit itself, Claude re-implemented the RGL’s paradigm selector in Python and compared: **225 of 1177 nouns (19%)** fell back to a wrong declension — two RGL bugs, affecting every Czech grammar.
- A human question: “is that actually an imperative?”

Task 2: Czech — outcome

e81159b, f756218 (10 July), 114fbd4, ab7e423 (16 July)

- InformathCze compiles and links; its own verb conjugator for *-ovat*, *-at*, *-it/-et/-ět*.
- 2003 terms, nouns with nominative, genitive and gender; wrong fallbacks from 225 to 1 after 7 new paradigms.
- 789 words in Godement's register; three-place verbs after an RGL fix.

Remaining problems

- *existuje matice M, takový, že* — should be *taková*: “such that” cannot agree in gender.
- Multi-word nouns decline only the head; “imperatives” are present tense.
- UserExtensionsCze still disabled; ~1500 terms unreviewed.

Task 3: Polish — why

- The second Slavic language, now with Czech as the model.
- The Polish RGL was in good shape (only 6 missing rules) — the problem turned out to be *nouns*.
- A test of whether the lessons of Czech generalize.

Task 3: Polish — prompt and interaction

“Create the Polish concrete syntax modules, following the Cze structure, so that Informath-Pol.gf compiles.”

and then, on request, Mizar words and user extensions.

- Three dead ends with nouns:
 - mkN guessed *zbióra, elementowie, przypadeka*;
 - the two-argument mkN dropped the genitive; fixing it gave *liczbaa* — reverted and documented;
 - the 1057 SGJP tables bake in an example word: *samolot* turns *wektor* into *wektot*.
- So Claude wrote its own declensions, taking the stem from the genitive.
- 12 subagents; they *refused* to fake genitives for adjectival nouns, which led to a new paradigm.
- Random generation found *dla wszystkich płytki*; a test of `meet_Verb2` found a word-order bug in the RGL.

Task 3: Polish — outcome

6ee105f, c735985 (16 July)

- InformathPol links: 2003 terms, 0 rejects; *niech x będzie grupą*.
- Mizar words reuse Wikidata terms where the sense matches: *pole relacji*, not *ciało*.
- The session log ends with six lessons “for the next language”.

Remaining problems

- *istnieje macierz M, taki, że* — the same agreement problem as Czech.
- *koł* for *kół*; weak terms (*pushforwardowy*).
- Needs a hand-compiled MarkupPol: `make next_grammar` fails on a fresh machine.

Task 4: French Wikidata — why and interaction

Why

- French was an official language, but WikidataWordsFre was hand-curated: 537 of 2003 terms, no verbs.

Prompt and interaction

“Complete WikidataWordsFre.gf so that every abstract WikidataWords fun has a French linearization.”

- A diff: 1481 missing (1008 nouns, 387 adjectives, 86 verbs).
- **15 parallel agents**, ~100 terms each; 15 name collisions dropped.
- A second pass on multi-word plurals: *trous spectraux*, *chemins de fer*.
- One smooth session.

Task 4: French Wikidata — outcome

9699638 (22 July): 2003/2003 terms, compiles.

```
lin close_Verb = mkVerb (mkV "fermer") ; ---Claude
```

- Every new line is marked ---Claude (1466 of them): `grep -v` gives a lexicon without them until they are checked.

Remaining problems

- Uncertain senses: *stack* → *champ*, *net* → *filet*, *quiver* → *carquois* (a quiver for arrows).
- ~21 invariant multi-word adjectives; source typos carried over (`minmum_Noun`).
- All unreviewed. Next candidates: German (~1650 missing), Swedish (~1850).

Part II: Godement's *Algebra*

The Godement project

- Roger Godement, *Algebra* (1968): a classic textbook, Part I (sets) and Part II (groups, rings, modules, linear algebra).
- Since 18 September, a companion repository `godement-index`:
 - the whole book as a UD treebank: 10 413 sentences;
 - a Dedukti formalization of Part I and §§ 6–24, with Claude, each constant linked to Godement's own sentence;
 - symbol tables, English readings, formulas, and an HTML index.
- The Informath commits of Part II were *needed by* this work: missing words, bugs found, new outputs, new grammar.

Task 5: Words for Godement — why and interaction

Why

- Every symbol-table entry must parse. Godement's wordings needed words the lexicon lacked: "*R* is *undecidable*", "*X* is *equipotent* to *Y*", "*a two-sided ideal*", "*a Cramer system*".

Prompt and interaction

"let us continue from PARTI-FORMALIZATION.md to complete the Godement book till the end of chapter 5"

- Claude added a word whenever a table entry failed to parse, and committed it with the formalization ("*make a commit if ready for it; I will push*").
- It deliberately did *not* add `countable_Adj`: already in WikidataWords, and a second declaration breaks the build.

Task 5: Words for Godement — outcome

583e0e5, a4427ce (24 Sept), bc4d431, 00c01cb, 6509853 (25 Sept)

```
fun 'two-sided_Adj' : Adj ;  
lin 'two-sided_Adj' = mkAdj "two-sided" ;
```

- 9 words: *undecidable*, *contradictory*, *equipotent*, *generate*, *two-sided*, *finite-dimensional*, *alternating*, *increasing*, *Cramer*.
- Symbol tables for Part I and §§ 6–24 build.

Remaining problems

- Words were the easy part: the *shapes* of entries were limited — no three-place nouns, no “into”, no negation (see Task 11).

Task 6: Clash-free macro names — why

- A LaTeX form $\$. . \$$ in a symbol table becomes a macro.
- Its name was the letters of the constant + MACRo: digits and underscores were dropped.
- card0, card1, card2 all became \cardMACRo; the last definition silently won.
- Nothing warned. The document compiled and said

$$2 \cdot x = 2 \quad \text{for} \quad 0 \cdot x = 0, \quad x^2 = x \quad \text{for} \quad x^1 = x.$$

Task 6: Clash-free macro names — prompt and interaction

- Found while reading Part I back. First: *“what are you doing?”* and *“I see that it stalls. Can you just jump over that?”*
- Claude proposed two fixes: spell digits as words (`\cardZeroMACRo`), or rename the constants.
“I know these fixes. They would not solve everything, because there can be names that already contain Zero etc. It would be better to generate all macro names with an algorithm that guarantees uniqueness. For instance, suffixes MACRoA, MACRoB, ... MACRoAA, MACRoAB, ...”
- Implemented as a bijective base-26 index over the whole table; *“this sounds good”*.
- Claude corrected its own diagnosis twice after measuring (the stall was one oversized theorem, not the flags).

Task 6: Clash-free macro names — outcome

f445edd (24 Sept)

	before	after
macros	<code>\cardMACRo</code> (3 constants)	<code>\cardMACRoY, ...Z, ...AA</code>
<code>mulCard_zero</code>	$2 \cdot x = 2$	$0 \cdot x = 0$
<code>powCard_one</code>	$x^2 = x$	$x^1 = x$
projections	$\text{pr}_2(x, y) = x$	$\text{pr}_1(x, y) = x$

Remaining problems

- Names depend on position: adding a LaTeX form renames later macros, so derived files must be rebuilt together.
- Informath has no regression test suite; checking is by comparing outputs.

Task 7: Proof texts — why

- Godement's theorems now had real Dedukti *proofs* (§ 6 onwards).
- Informath could print a proof as numbered lines, but:
 - every argument of a rule became a line, mostly “there is a proposition”: § 6 Theorem 1 gave 46 lines, 14 of them steps;
 - the output was a LaTeX 2.09 demo (`\documentstyle`) that did not compile;
 - lines were raw token lists: `$ y $ is equal to $ x $ .`
- Goal: proofs that a mathematician can read, as PDF.

Task 7: Proof texts — prompt and interaction

- No transcript survives for this day; the story is in four commits made within half an hour:
 1. show the steps, not the arguments (a heuristic filter);
 2. replace the heuristic by *profiles*: `#SHOW forallE [3,4]` in the symbol table, or read the premisses off the rule's type;
 3. a document that can be read and compiled;
 4. unlex the lines.
- Each was committed together with the regenerated proofs in `godement-index`.
- A slip: commit 2 swept in unrelated files (slides, logs, `*.gf~` backups).

Task 7: Proof texts — outcome

0623e2b, dabdd50, d183036, 2ef5f55 (25 Sept)

```
5. Assume that x is equal to y.  
6. Assume that y is equal to z.  
7. x is equal to y and y is equal to z.      andI 5, 6  
8. x is equal to z.                          ifE 4, 7
```

- PDFs of the proofs for §§ 6–10, 12–17, 22 (27 pages for § 6).

Remaining problems

- Each proof is headed by its raw Dedukti type, not an English theorem.
- § 6 Theorem 1 prints two lines: the declaration, then its *conclusion* as an assumption; the steps are lost.
- Proof labels leak into formulas (“the value of f at hy ”).

Task 8: Compressing hypotheses — why and interaction

Why

- Every variable had its own line: “Let x be ... Let y be ... Let z be ...” — long and unlike mathematical prose.

Prompt and interaction

“I want to improve `src/ProofText.hs` by compressing hypotheses. If there are consecutive proof lines introducing a variable with the same type, they should be compressed to one line using the abstract syntax function `GVarsHypo` on the list of those variables. The lines should of course be renumbered to reflect this.”

- Claude also found that the Dedukti table (`-proof-terms`) would no longer match, and that the flag had never been accepted at all.
- *“Do compress the Dedukti table. But where in the code are the wrong variable names created?”* — diagnosed (beta reduction under the rule’s binder), fix proposed, not made.

Task 8: Compressing hypotheses — outcome

c391de6 (26 Sept)

Before (8 lines)

1. Let x be a mathematical object.
2. Let y be a mathematical object.
3. Let z be a mathematical object.
4. If x is equal to y and ...

After (6 lines)

1. Let x , y and z be mathematical objects.
2. If $x = y$ and $y = z$, then $x = z$.
3. Assume that $x = y$.

- The symbolic $x = y$ came from another change; asked about it: *“Yes, I did want to increase the usage of symbolic expressions instead of verbal ones.”* — saved as a lasting preference.

Remaining problems

- Wrong bound-variable names (“for all elements x and m of $\mathbb{N}$ ”) are still there.

Task 9: Standard logical notation — why and interaction

Why

- Readers want each statement also as a compact formula ($\forall, \wedge, \rightarrow, \vdash$), next to the English.
- Informath had symbolic notations for constants, but not for the logic.

Prompt and interaction

“I would like to have grammars/MathCoreLatex.gf and InformathLatex that generates formulas with standard logical notation and uses the symbolic notations defined in SymbolicConstants-Latex and in .dkgf files. In RunInformath, the flag -to-symbolic-latex should translate Dedukti to this notation. [...] Use the precedences in the same way as in TermsLatex.”

- Claude chose conventions itself and offered to change them: sequents $\Gamma \vdash A$, $:\Leftrightarrow$ for definitions, no proofs shown.
- *“Also update godement-index.html with these formulas.”*

Task 9: Standard logical notation — outcome

f87c660 (26 Sept): three new GF modules, a flag, a Makefile target.

- English: “Let f be a function from mathematical objects to mathematical objects. Let u and v be mathematical objects. Assume that $u = v$. Then $f(u) = f(v)$.”
- Formula: $f \in \text{Obj} \rightarrow \text{Obj}, u, v \in \text{Obj}, u = v \vdash f(u) = f(v)$
- All 1138 declarations in 5 seconds; a “Formula” fold in the index.

Remaining problems

- Constants without notation print as `isGroup( $g$ )`.
- Conjunctions nest with explicit brackets: $A \wedge (B \wedge (C \wedge \dots))$.
- Claude’s conventions not yet confirmed; long formulas only scroll.

Task 10: GodementSyntax — why

- Informath's grammar covers the statements that come *out* of Dedukti.
- Godement *writes* differently. Counted in the 599 formalized sentences (UD trees, regex + dependency patterns):

Construction	sentences	in the grammar?
" x is said to be A if ..."	53	no
" x is called the N of y "	65	no
"for x to be A it is necessary and sufficient that"	24	no
"the following conditions are equivalent: a) ... b) ..."	33	no
"there exists a unique x such that"	22	no
"subgroup of G which contains B " (relative clause)	105	no
"if P then Q ", "for all x ", "there exists"	180+	yes

Task 10: GodementSyntax — prompt and interaction

“Let us investigate the linguistic structures used in definitions and theorem statements in Godement. [...] Can you make a list of the structures of statements in Godement by using the Informath categories Prop, Kind, Exp, Adj, etc? [...] Do not try to run GF in parsing mode yet, because it can be slow.”

“Excellent! Can you suggest GF abstract syntax functions, linearizations using RGL, and mappings to MathCore”

“You can add GodementSyntax to UserExtensions and a test case [...] and a Makefile entry for godement_syntax that produces a pdf and opens it.”

- Parsing real sentences revealed what linearizing did not: $\$ \dots \$$ is indexed before parsing, a hidden argument crashes the parser, a comma too many blocks a parse.

Task 10: GodementSyntax — outcome

865eae0 (26 Sept): about 50 functions; semantics in Haskell; a flag `-translate-core`; make `godement_syntax` $\rightarrow$ PDF.

- *For an element x of G to be reflexible, it is necessary and sufficient that x has a left reflection.*
 $\Rightarrow$ for all elements x of G , (x is reflexible, if and only if there is a left reflection of x).
- *There exists a unique element x of G such that x is invertible.*
 $\Rightarrow$ there exists $x \dots$ and for all $y \dots$, if y is invertible, then $y = x$.
- 38 of 38 test sentences parse; no regressions on the old tests.

Remaining problems

- Ambiguity: several readings per sentence; ugly fresh variables `_g0`.
- English only; “ x is a K ” has no Dedukti translation when K is a type.
- Not yet run on Godement’s actual 10 413 sentences.

Task 11: GodementExamples — why

- Symbol tables speak Godement only as far as Examples.gf lets them.
- The formalization notes (written with Claude) listed the gaps:
 - no three-place noun: “a homomorphism of G into H ” became “ f is a homomorphism”;
 - no “into”, “onto”, “generated by”;
 - no negation: “ x is not an element of y ”;
 - “zero” and “one” but no “two”.
- 216 table entries showed fewer arguments than their constants had.

Task 11: GodementExamples — prompt and interaction

“We noticed that some syntactic forms that should be usable in .dkgf entries are missing from the grammar. [...] I would like to start with a new module GodementExamples [...] You can try to rebuild the dkgf files by using these new functions where necessary. If this means that you need some new categories, add them and their application functions also to GodementExamples.”

- New categories need more than grammar: the Dedukti $\leftrightarrow$ MathCore conversions in Haskell had to learn them.
- Verified by reading chapters back with old and new tables and diffing.
- “go on”: a second round of rewordings, and the whole index rebuilt.

Task 11: GodementExamples — outcome

35184d0 (26 Sept): 3 new categories, 5 example forms, 12 prepositions.

before	after
"#1 is a homomorphism" (not worded)	"#1 is a homomorphism of #2 into #3"
"the class of #3 under #2"	"the subgroup of #1 generated by #2"
"the class of #3 modulo #2"	"the class of #3 modulo #2 in #1"
"#1 $\notin$ #2" only	"#1 is not an element of #2"

Remaining problems

- “the set of all subsets of X with n elements” still does not parse.
- Negated definitions read as tautologies: “ $x \notin y$ if $x \notin y$ ”.
- The next morning: English readings of §§ 10–24 had never been possible; 21 declarations are too big to word in 30 seconds.

Observations and the future

What we observed

Worked well

- Bulk lexical work with parallel subagents (2000 terms in minutes).
- Tireless crash-driven iteration (Czech RGL).
- Self-audits *when asked*: the Czech paradigm check.
- Grounding claims in measurements and diffs.
- Code in the style of the surrounding code.

Needed a human

- Design decisions: unique macro names, symbolic preference, “just time out and move on”.
- linking ... OK $\neq$ correct.
- Careless moments: files swept into a commit, an installed binary overwritten mid-comparison.
- Native-speaker judgement: none of the new words is reviewed.

The division of labour: I set direction and judge; Claude explores, builds, measures.

Speculation: what would be good to do next

- **Review:** native checking of the Finnish, Czech, Polish and French words; gender agreement of “such that” in Slavic languages.
- **Tests:** a regression suite and CI — today regressions are found by diffing outputs by hand.
- **Scale:** faster or incremental search for wordings, so that big statements (determinants of order 3) get English too.
- **Proofs:** repair truncated proofs, English theorem headings, proper bound-variable names.
- **Coverage:** run GodementSyntax on all 10413 sentences and measure how much of the book parses; port it to French, German, Swedish.
- **Autoformalization:** parse Godement’s own text into Dedukti and type-check it against the hand-made formalization.
- **Hybrid:** an LLM proposes symbol-table entries and grammar rules, the grammar and the type checker verify them.

Thank you!

`github.com/GrammaticalFramework/informath`